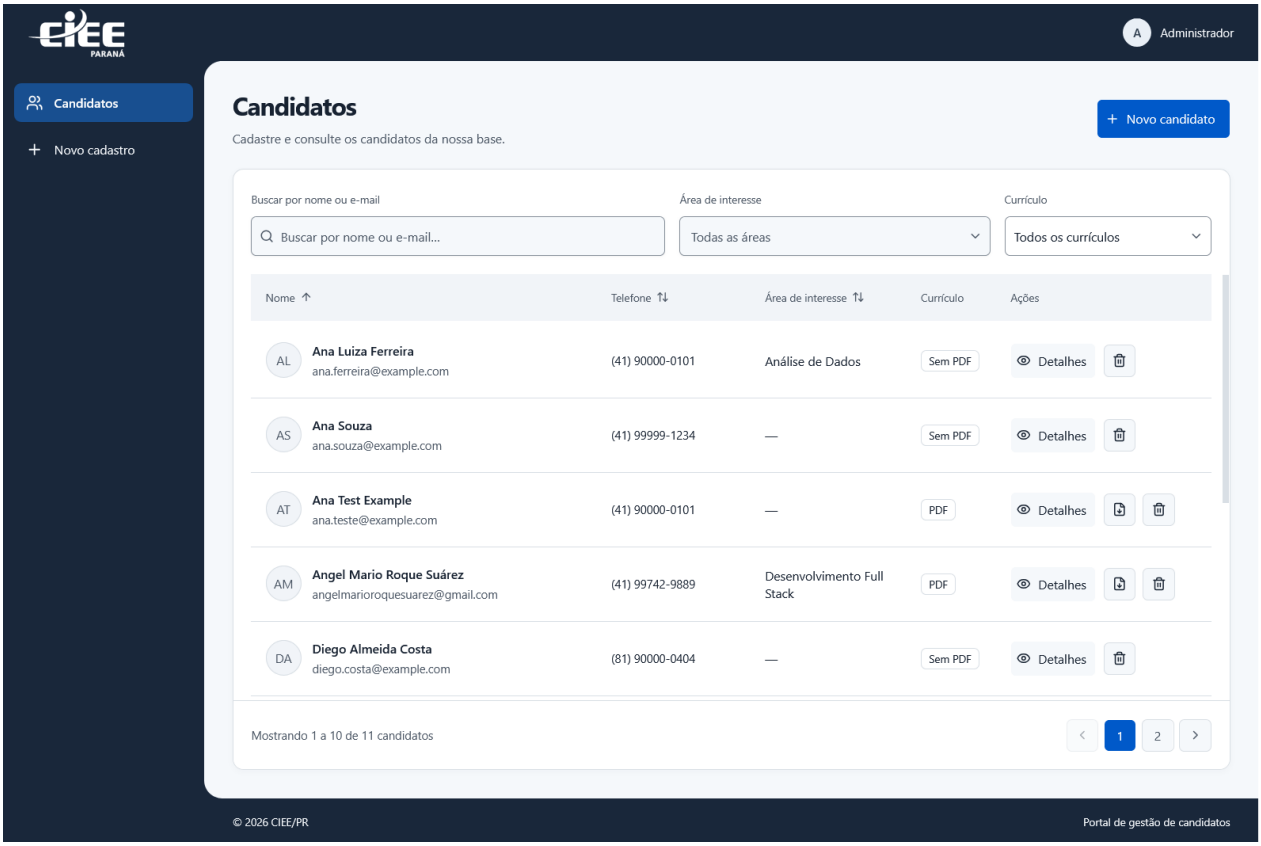
DESAFIO TÉCNICO / CIEE/PR

Cadastro de Currículos

Implementação e decisões técnicas

Uma aplicação para cadastrar, revisar e consultar candidatos, com preenchimento manual ou assistido por PDF.



Interface real da solução: busca, filtros, ordenação e paginação.

Angel Mario Roque Suárez
Desenvolvedor Fullstack
React / Node.js / SQL Server

Curitiba – PR
3 de outubro de 2026
Versão 1.0.0

Sumário

1. Visão geral da solução 1
2. Atendimento ao desafio 2
3. Interface e experiência de uso 3
4. Arquitetura e decisões técnicas 6
5. Verificação e qualidade 8
6. Processo de desenvolvimento 9
7. Roteiro de avaliação 10
A. Diagramas da implementação 11

Guia de leitura

Para conhecer a entrega, comece pela visão geral e pelas telas da Seção 3. As decisões estão na Seção 4; os cenários e relatórios, na Seção 5. O roteiro de avaliação orienta a execução e o anexo reúne os 13 diagramas.

1. Visão geral da solução

Desenvolvi esta aplicação para o desafio técnico de Desenvolvedor de Sistemas do CIEE/PR. A solução reúne os dois caminhos de cadastro solicitados, persistência no SQL Server, listagem e consulta de detalhes, com uma interface integrada à API.

O cadastro manual e o assistido por PDF utilizam o mesmo formulário e as mesmas validações. A leitura do currículo acontece no backend e retorna sugestões de nome, e-mail e telefone. O usuário revisa os dados, complementa as informações e confirma o cadastro. O PDF é opcional em todo o processo.

Fluxo principal
Preencher ou importar → revisar → salvar → consultar os detalhes.

1.1. Melhorias incorporadas

Além do fluxo central, implementei recursos que facilitam o trabalho de consulta e organização dos candidatos:

- busca por nome ou e-mail, filtros por área e presença de currículo;
- ordenação e paginação, com layout adaptável em tabela ou cartões;
- armazenamento opcional do PDF e download pela tela de detalhes;
- exclusão individual com confirmação e operação em lote pela API;
- e-mail único e normalizado, telefone brasileiro com DDD e confirmação antes de abandonar alterações do formulário;
- documentação interativa da API e execução integrada com Docker Compose.

1.2. Tecnologias

Camada	Tecnologias utilizadas
Interface	React 19.2.8, TypeScript, Vite 8.2.2, TanStack Router 1.170.32 e TanStack Query 5.104.0.
API	Node.js 22+, NestJS 12.0.1 e `pdf-parse` 2.4.5.
Dados	SQL Server 2022, Prisma 7.10.0 e migrations versionadas.
Contratos	Zod 4.5.4 compartilhado entre frontend e backend.
Arquivos	RustFS compatível com S3 no ambiente local.
Workspace	pnpm 11.8.0, Turborepo e Ultracite.

2. Atendimento ao desafio

O quadro relaciona os requisitos do enunciado ao comportamento entregue. Os diagramas do anexo detalham as operações e suas integrações.

Requisito	Solução implementada
Cadastro manual	Formulário com nome e e-mail obrigatórios e telefone, área e resumo opcionais, sem exigir documento.
Cadastro com PDF	Leitura no backend e sugestões para revisão no mesmo formulário do cadastro manual.
Revisão dos dados	Campos editáveis antes da confirmação; informações ausentes podem ser preenchidas manualmente.
PDF opcional	A falta do arquivo ou falha de leitura mantém o cadastro manual disponível. Guardar o PDF é uma opção separada.
Validação	Contratos compartilhados, verificação de e-mail e validação de arquivo PDF com limite de 5.000.000 bytes.
Persistência	Cadastro e consultas pela API, com Prisma, SQL Server e migrations para criar a estrutura.
Listagem e detalhes	Candidato disponível na listagem após salvar, com acesso a uma tela de dados completos.
Mensagens claras	Feedback para validação, arquivo inválido, falha de leitura, cadastro salvo e registro ausente.
Configuração e avaliação	README com versões e comandos, exemplos de ambiente, testes e currículo fictício para importação.
Registro do trabalho	Relato de decisões, uso de IA, verificações e tempo em `DESENVOLVIMENTO.md`.

2.1. Campos e consistência dos dados

Campo	Regra aplicada
Nome completo	Obrigatório; até 200 caracteres, sem aceitar apenas espaços.
E-mail	Obrigatório, formato válido, até 254 caracteres; normalizado para minúsculas e único no banco.
Telefone	Opcional; telefone brasileiro com DDD, armazenado em dígitos nacionais e apresentado com prefixo +55.
Área ou cargo	Opcional; aceita apenas opções da lista predefinida compartilhada entre interface e API.
Resumo profissional	Opcional; até 2.000 caracteres.

3. Interface e experiência de uso

Organizei as telas em torno do cadastro e da consulta. A listagem reúne busca, filtros e ações; o formulário apresenta a importação como auxílio ao preenchimento; os detalhes agrupam os dados e o acesso ao currículo. As imagens são capturas da aplicação em execução.

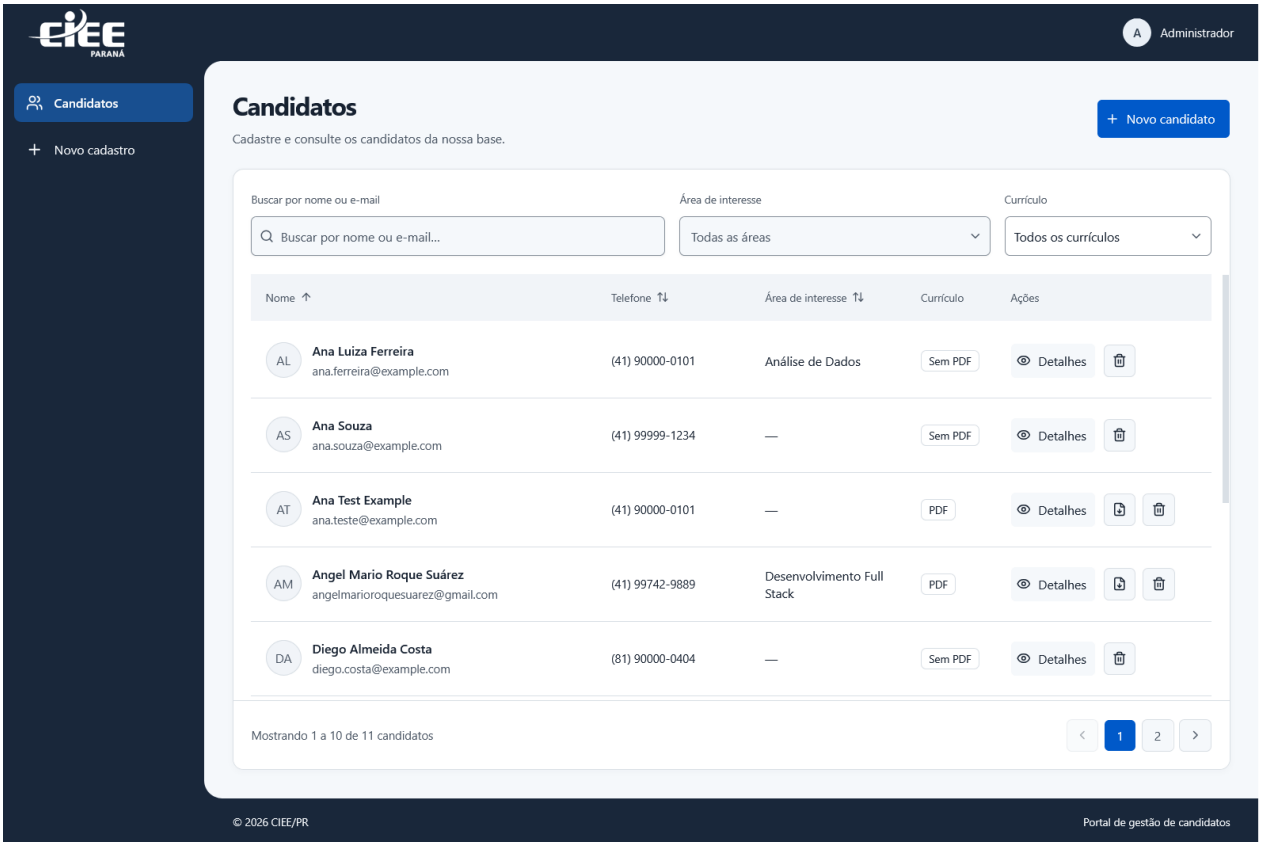


Figura 1: Listagem com busca, filtros, ordenação, ações e paginação.

Os filtros são mantidos na URL, facilitando retornar à consulta. Conforme o espaço horizontal disponível, os candidatos são apresentados em tabela ou cartões. A ausência de resultados possui mensagem e ação para limpar a busca.

3.1. Cadastro assistido por PDF

A importação preenche sugestões no formulário compartilhado. O usuário confere os campos e escolhe se deseja guardar o arquivo junto ao cadastro. Após salvar, a interface abre os detalhes do candidato.

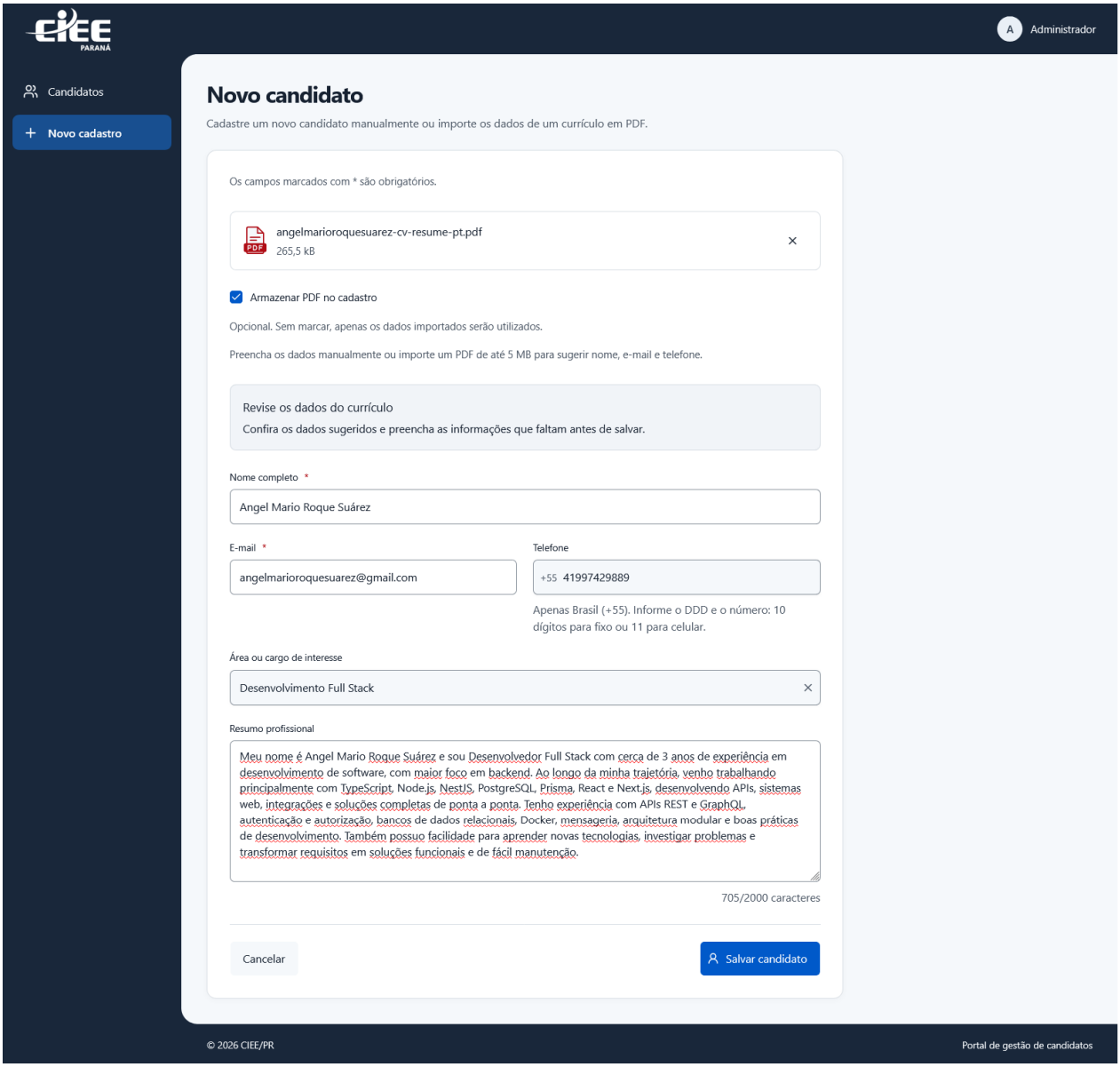


Figura 2: Formulário com revisão das sugestões e armazenamento opcional do currículo.

3.2. Detalhes do candidato

A tela apresenta dados de contato, área de interesse, resumo profissional e informações do arquivo quando ele foi armazenado. A ação de download permite acessar o currículo.

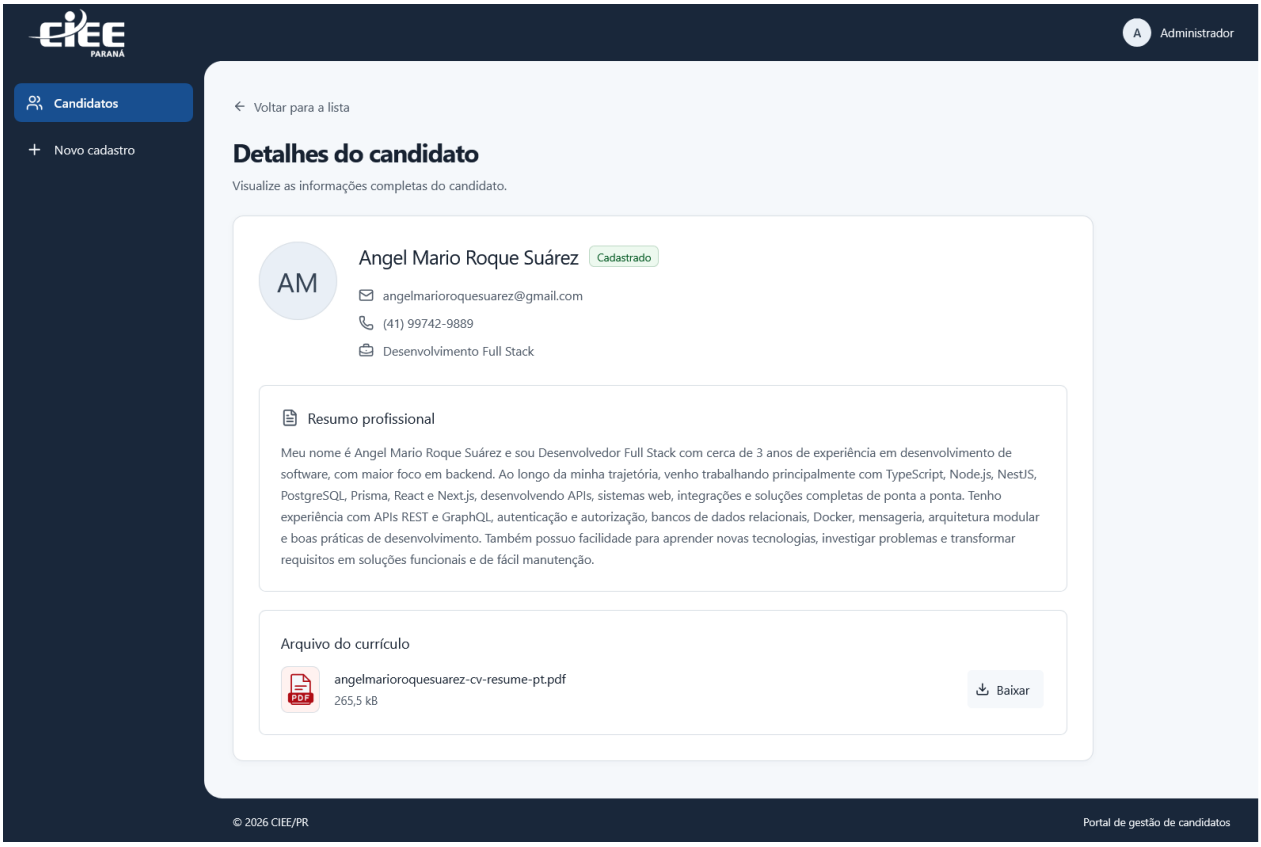


Figura 3: Dados do candidato e acesso ao currículo na tela de detalhes.

4. Arquitetura e decisões técnicas

4.1. Organização do código

O monorepo separa a interface em `apps/web`, a API em `apps/backend` e os contratos em `packages/shared`. No frontend, componentes e hooks organizam os campos, a importação e a confirmação. No backend, controllers recebem as requisições e serviços concentram as operações de candidatos, extração e armazenamento.

Decisão	Motivo e efeito
Contratos compartilhados	Manter as mesmas regras no formulário e na API, incluindo validação das respostas recebidas.
Importação sem persistência	Permitir revisar ou descartar sugestões sem criar registros ou arquivos antecipadamente.
Prisma e migrations	Representar o modelo de dados e reproduzir sua estrutura no SQL Server.
TanStack Query	Gerenciar consultas, cancelamento e atualização do cache após cadastro ou exclusão.
Armazenamento separado	Guardar o PDF em S3/RustFS e manter seus metadados associados ao candidato no banco.
Docker Compose	Inicializar os serviços, preparar o armazenamento e aplicar migrations no ambiente de avaliação.

4.2. Modelo de dados

`candidates` armazena identificador e os cinco campos do formulário. `curriculum_files` guarda os metadados de zero ou um PDF por candidato: identificador, referência ao candidato, chave de armazenamento, nome original, tipo, tamanho e data do upload. A referência ao candidato é única e utiliza exclusão em cascata dos metadados no SQL Server. O serviço coordena a remoção do objeto no armazenamento. O e-mail possui restrição única também no banco.

4.3. Importação, revisão e armazenamento

O backend valida o limite e a assinatura do PDF e usa `pdf-parse` para extrair texto. A resposta contém sugestões, campos ausentes e avisos. Sugestões encontradas atualizam nome, e-mail e telefone; valores ausentes preservam o preenchimento atual. Área e resumo são completados manualmente.

O arquivo só acompanha a confirmação se a opção de guardá-lo estiver marcada. Nesse caso, o backend envia o objeto com uma chave UUID e cria candidato e metadados com Prisma. Se SQL falhar, tenta remover o upload. O download retorna uma URL pública do ambiente local, acessada diretamente pelo navegador.

4.4. API e comportamento das operações

Método	Rota sob /api	Operação
POST	`/candidates`	Cadastro em JSON ou multipart com PDF opcional.
POST	`/candidates/import-pdf`	Extração de sugestões sem persistência.
GET	`/candidates`	Busca, filtros, ordenação e paginação.
GET	`/candidates/:id`	Dados e metadados do currículo.
GET	`/candidates/:id/curriculum/download`	URL para acessar o arquivo.
DELETE	`/candidates/:id`	Exclusão individual.
DELETE	`/candidates`	Exclusão de até 100 IDs pela API.

A busca considera nome ou e-mail. Os filtros incluem área e presença de currículo. A ordenação aceita identificador, nome, e-mail, telefone ou área. A paginação usa 10 itens por padrão e até 100 por página; a resposta informa itens, total e metadados da página. A operação em lote valida e deduplica IDs e informa a quantidade removida.

4.5. Tratamento de erros e confirmação

As operações distinguem dados inválidos (400), registro ausente (404), e-mail duplicado (409), arquivo grande (413), tipo inválido (415), falha de leitura (422) e armazenamento indisponível (503). As mensagens permitem identificar o problema e continuar o cadastro manual quando necessário.

A interface confirma exclusões e a saída de um formulário com alterações. Ao trocar o PDF, cancela a importação anterior e descarta respostas tardias. A exclusão remove primeiro o arquivo e depois o registro; a compensação do upload no cadastro e essa ordem de remoção tratam falhas entre serviços.

4.6. Ambiente local

O Compose serve o frontend por Nginx em `localhost:3001`, a API em `localhost:3000`, SQL Server na porta 1433 e RustFS nas portas 9000 (S3) e 9001 (console). A referência interativa da API está em `/docs`, com contrato JSON em `/openapi.json`. O README descreve a configuração e os comandos para iniciar a stack.

5. Verificação e qualidade

Durante o desenvolvimento, utilizei verificações de tipos, compilação, lint e formatação, testes unitários e testes e2e da API. A integração utiliza SQL Server e RustFS reais em um banco dedicado com sufixo `_test`. A suíte cria dados próprios e remove os candidatos que produz.

Grupo de cenários	Verificação realizada no desenvolvimento
Cadastro e validação	Campos obrigatórios, formatos, normalização, duplicidade e persistência.
Importação de PDF	Sugestões, arquivo fictício, campos ausentes, arquivos inválidos e tamanho excedido.
Consulta	Listagem, filtros, paginação e consulta de detalhes pela API.
Currículo	Upload, download, exclusão e tratamento de falhas entre serviços.
Interface	Testes de chamadas da API, cache e hooks; revisão no navegador do formulário e das confirmações.
Contrato da API	Schemas OpenAPI de operações e respostas.

5.1. Relatórios de cobertura

Os relatórios existentes de 3 de outubro de 2026 registram a seguinte cobertura de linhas. As suítes possuem medições independentes.

Suíte	Cobertura de linhas
Backend – testes unitários	52,39%
Backend – integração/e2e	78,96%
Frontend	38,35%

As medições incluem fontes não exercitados pelos testes, conforme a configuração das aplicações. Os arquivos `coverage-summary.json` e relatórios HTML permitem conferir os resultados. A automação de interações completas entre páginas é uma evolução dos testes de frontend, que hoje cobrem contratos, cache e hooks.

5.2. Reprodução

Os comandos de tipos, compilação, lint, testes e cobertura estão no README. `QUALIDADE.md` descreve os relatórios; o README do backend explica a preparação do banco dedicado. Este documento consolida os registros do desenvolvimento; a revisão editorial não executou novamente as suítes.

6. Processo de desenvolvimento

6.1. Planejamento e execução

Antes da programação, dediquei aproximadamente 2 horas à modelagem no Visual Paradigm e à preparação do LaTeX. Organizei o trabalho por contratos, persistência, cadastro, importação, consultas e revisão. Ao final, dediquei aproximadamente 1 hora à sincronização dos diagramas e da documentação com a implementação, reunindo os 13 modelos apresentados no anexo.

6.2. Uso de IA e decisões próprias

Utilizei Codex no T3 Code, com GPT-6.1-Sol nas sessões registradas, como apoio à análise, implementação, revisão e documentação. Defini as prioridades e orientei os ajustes de validação, organização do formulário e comportamento da aplicação.

Pedidos de unificação das regras resultaram nos contratos compartilhados; a revisão do formulário ajudou a separar componentes e hooks; a revisão da API alinhou os testes ao contrato paginado e completou os schemas OpenAPI. Adaptei as sugestões ao projeto e mantive o formulário com estado React e validação Zod. O relato completo está em `DESENVOLVIMENTO.md`.

6.3. Tempo aproximado

Etapa	Tempo
Modelagem e preparação do LaTeX antes de programar	2h
Desenvolvimento em 02/10: 14h30–17h20 e 19h30–23h	6h20
Desenvolvimento em 03/10: 08h30–12h	3h30
Atualização final dos diagramas e da documentação	1h
Enriquecimento da documentação e revisão final em 03/10: 13h30–14h30	1h
Total dos períodos registrados	**13h50**

6.4. Limites e evolução

A extração trabalha com texto disponível no PDF. Documentos digitalizados e layouts ambíguos podem exigir preenchimento manual, como previsto no desafio. A revisão dos dados permanece disponível em todos os casos.

Com mais tempo, ampliaria a automação das interações da interface e a reconciliação entre banco e armazenamento. Para um contexto com dados reais, adotaria controle de acesso e URLs temporárias para os currículos; o ambiente entregue utiliza acesso direto para avaliação local.

7. Roteiro de avaliação

7.1. Executar a aplicação

Na raiz do repositório, crie `apps/backend/.env` a partir de `apps/backend/.env.example`, configure a senha do SQL Server e execute:

```
docker compose -env-file apps/backend/.env up -d -build
```

O Compose inicia os serviços, prepara o bucket e aplica as migrations. Abra `http://localhost:3001`. O README também oferece o fluxo de desenvolvimento local e os comandos de testes.

7.2. Explorar os dois caminhos de cadastro

1. Cadastre um candidato manualmente, sem arquivo, e consulte seus detalhes.
2. Abra um novo cadastro e importe `examples/curriculo-ficticio.pdf`.
3. Revise as sugestões e complete área e resumo. Marque a opção de guardar o PDF se desejar experimentar o armazenamento.
4. Salve, consulte os detalhes e acesse o currículo quando armazenado.
5. Volte à listagem e experimente busca, filtros, ordenação e paginação.

7.3. Materiais para consulta

Material	Conteúdo
`README.md`	Visão da aplicação, capturas reais, versões, configuração e comandos.
`DESENVOLVIMENTO.md`	Organização, decisões, participação da IA e tempo dedicado.
`QUALIDADE.md`	Cenários verificados e relatórios de cobertura.
`apps/backend/README.md`	API, ambiente, armazenamento e banco dedicado para testes.
`examples/`	Currículo fictício e exemplos para importação.
Anexo deste documento	Casos de uso, arquitetura, dados, implantação e sequências.

Resultado entregue

Cadastro manual e assistido por PDF com revisão, persistência no SQL Server e consulta de candidatos, acompanhado de recursos de organização, testes e documentação para avaliação.

A. Diagramas da implementação

Os modelos foram preparados no Visual Paradigm e atualizados conforme o desenvolvimento. Este anexo reúne as 13 imagens exportadas do projeto nativo `visualp.vpp`. Cada prancha apresenta o propósito do diagrama e o fluxo correspondente.

ID	Diagrama	Referência
D01	Casos de uso	Figura 4
D02	Componentes e integrações	Figura 5
D03	Modelo de dados SQL Server	Figura 6
D04	Importação e revisão de PDF	Figura 7
D05	Implantação local com Compose	Figura 8
D06	Cadastro manual	Figura 9
D07	Consulta filtrada e paginada	Figura 10
D08	Consulta de detalhes	Figura 11
D09	Cadastro com arquivo opcional	Figura 12
D10	Download do currículo	Figura 13
D11	Exclusão individual	Figura 14
D12	Exclusão em lote pela API	Figura 15
D13	Descarte de alterações	Figura 16

D01 – Casos de uso

O usuário cadastra, revisa e consulta candidatos e acessa seus currículos. O cliente da API também pode realizar exclusões em lote. Os dois caminhos de cadastro compartilham o formulário e as regras de validação.

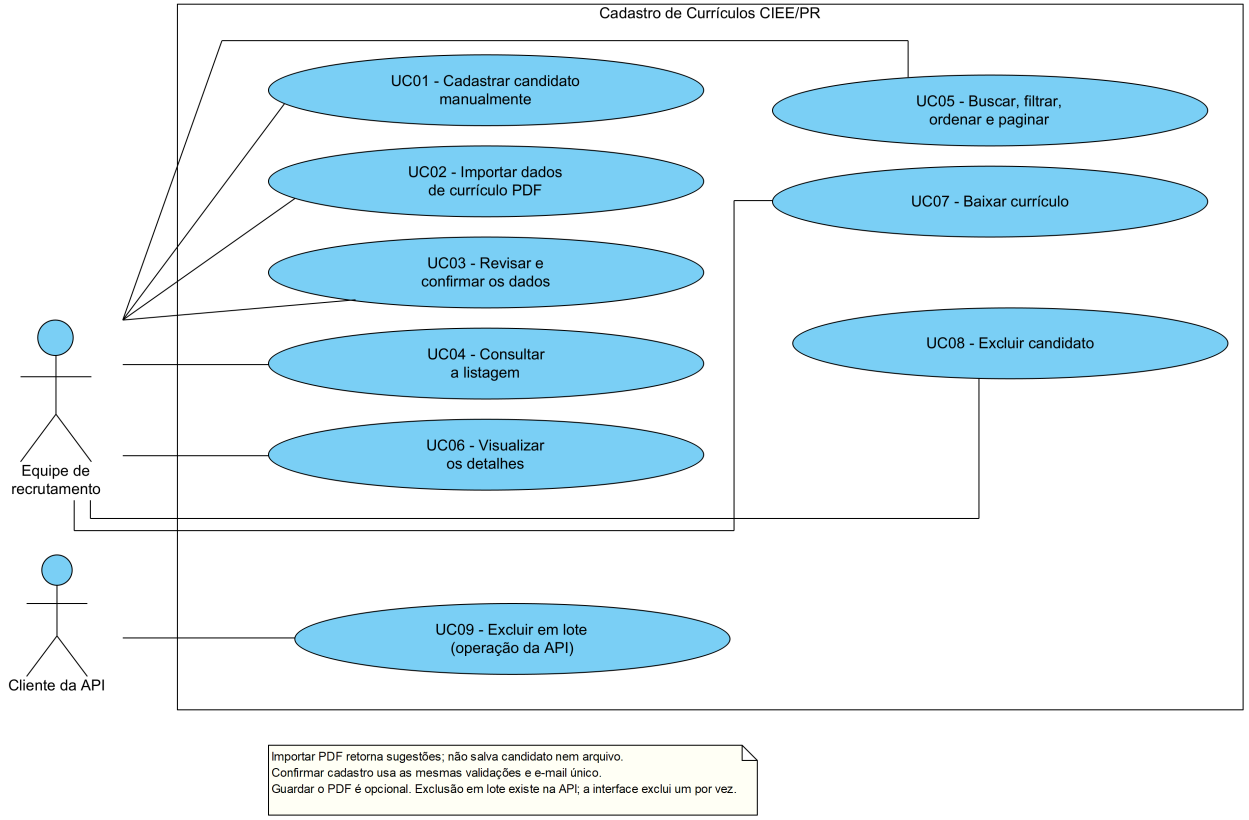


Figura 4: D01 – Casos de uso

D02 – Componentes e integrações

A interface React consulta a API NestJS. Os serviços separam candidatos, extração de PDF, acesso ao SQL Server com Prisma e operações no armazenamento S3/RustFS. O navegador acessa diretamente o arquivo ao receber sua URL.

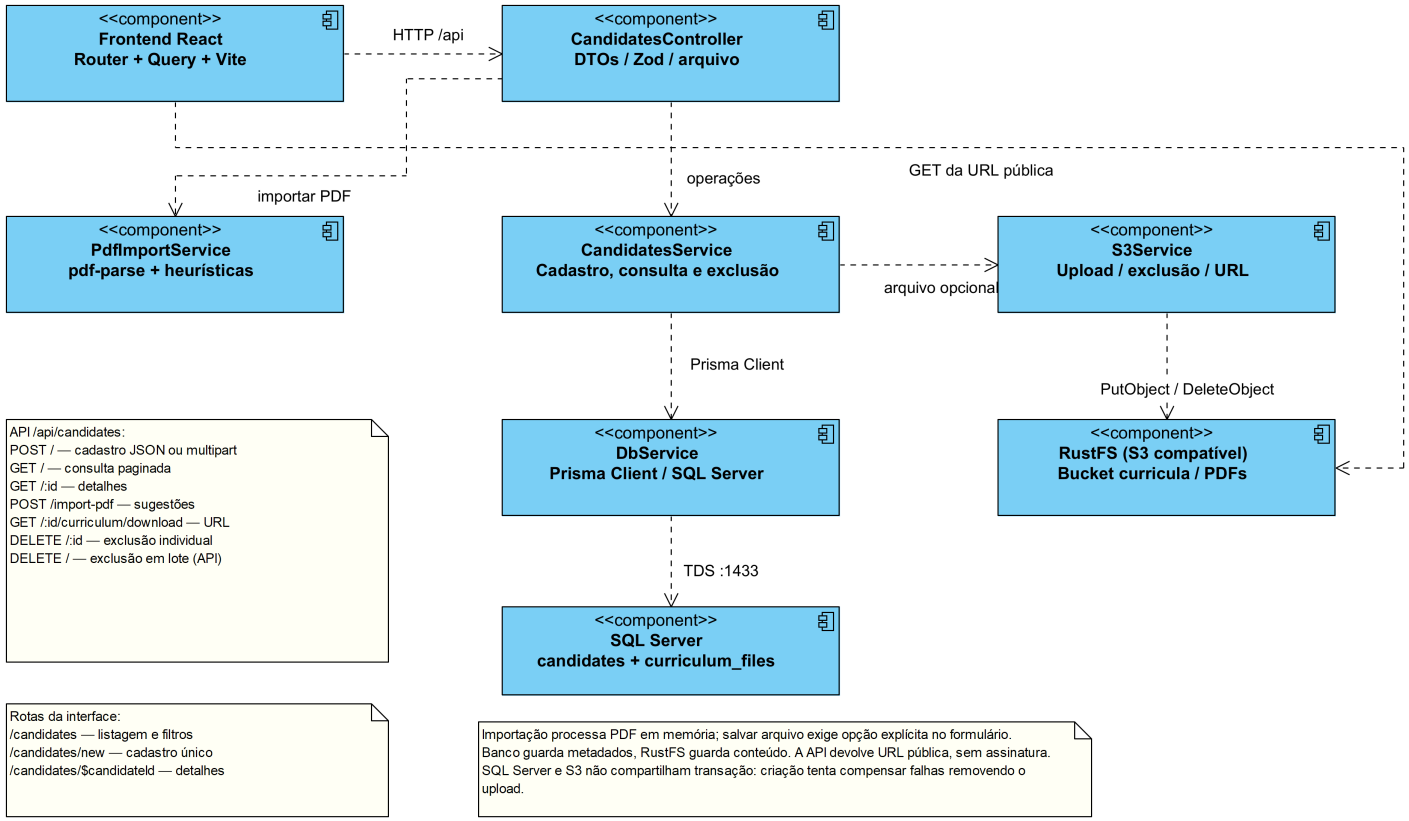


Figura 5: D02 – Componentes e integrações

D03 – Modelo de dados SQL Server

Um candidato possui zero ou um arquivo. O e-mail é único e a referência ao candidato em curriculum_files também é única. A cascata SQL remove os metadados; o serviço coordena a remoção do objeto no armazenamento.

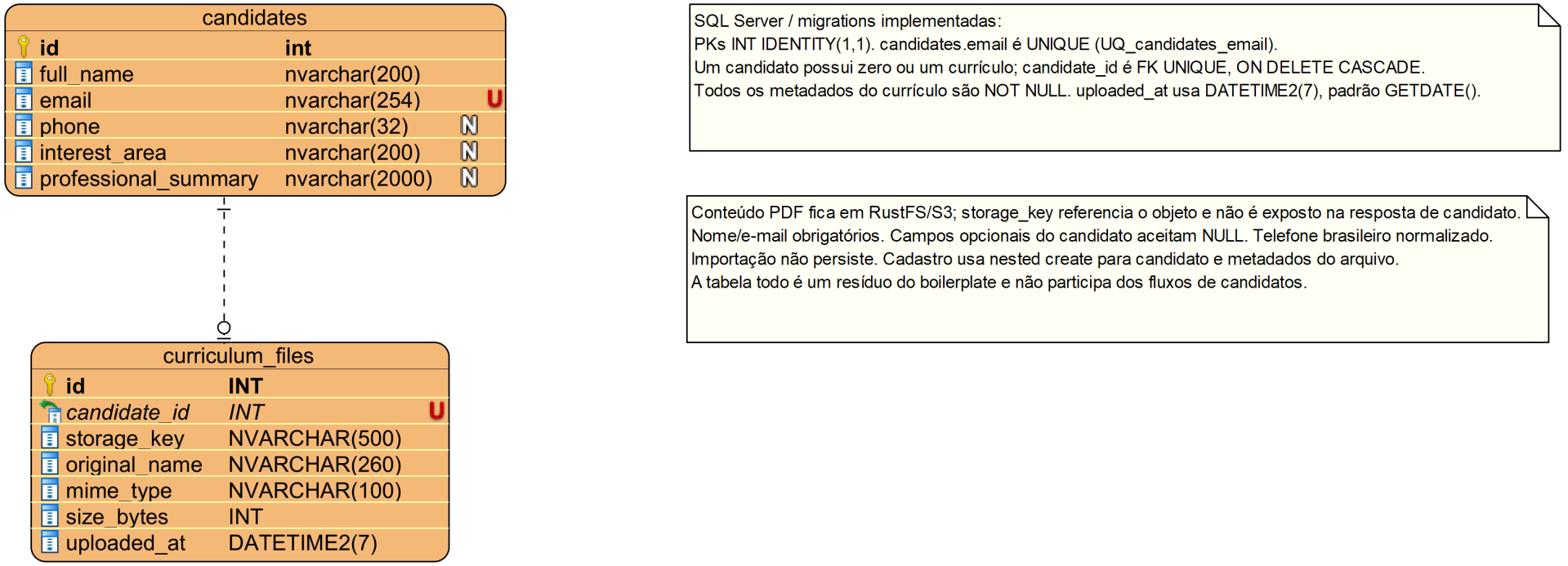


Figura 6: D03 – Modelo de dados SQL Server

D04 – Importação e revisão de PDF

A API valida e lê o PDF em memória e retorna sugestões, campos ausentes e avisos. A interface permite revisar o resultado. Essa operação não cria candidatos nem guarda arquivos, e o cadastro manual continua disponível.

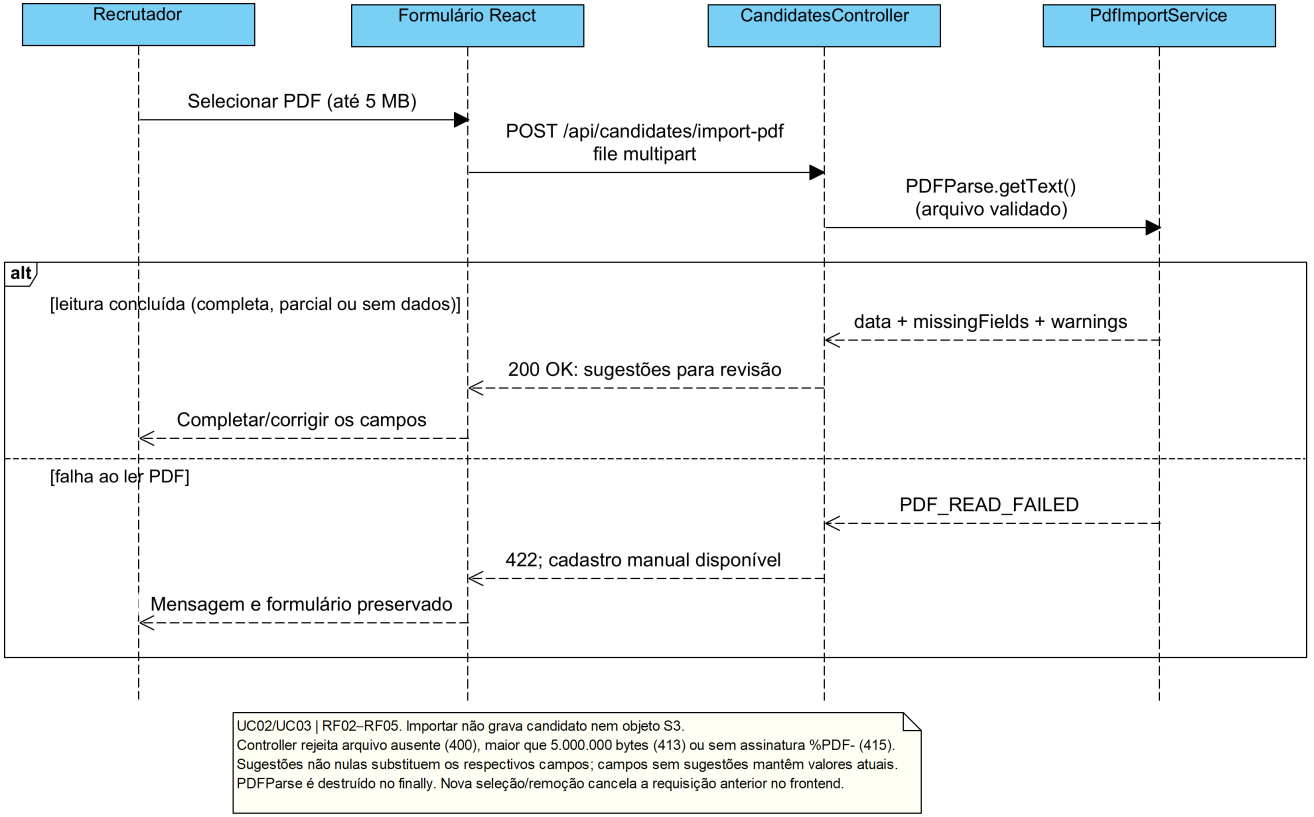


Figura 7: D04 – Importação e revisão de PDF

D05 – Implantação local com Compose

O ambiente reúne frontend servido por Nginx, API NestJS, SQL Server e RustFS. O Compose coordena inicialização do bucket, migrations e volumes. As portas locais permitem avaliar a interface, a API e o armazenamento.

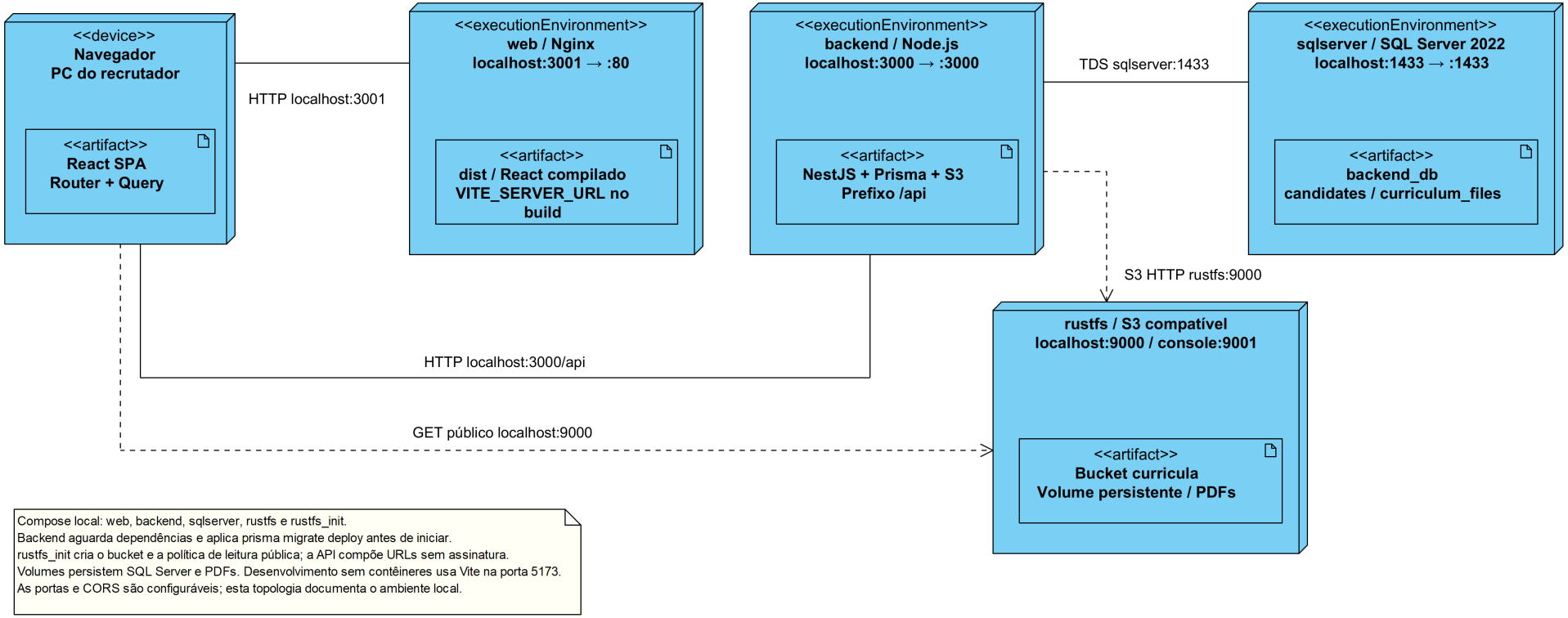


Figura 8: D05 – Implantação local com Compose

D06 – Cadastro manual

O formulário envia os dados sem exigir PDF. A API valida, normaliza o e-mail e persiste o candidato, com proteção contra duplicidade. Após salvar, a interface atualiza o cache e abre a tela de detalhes.

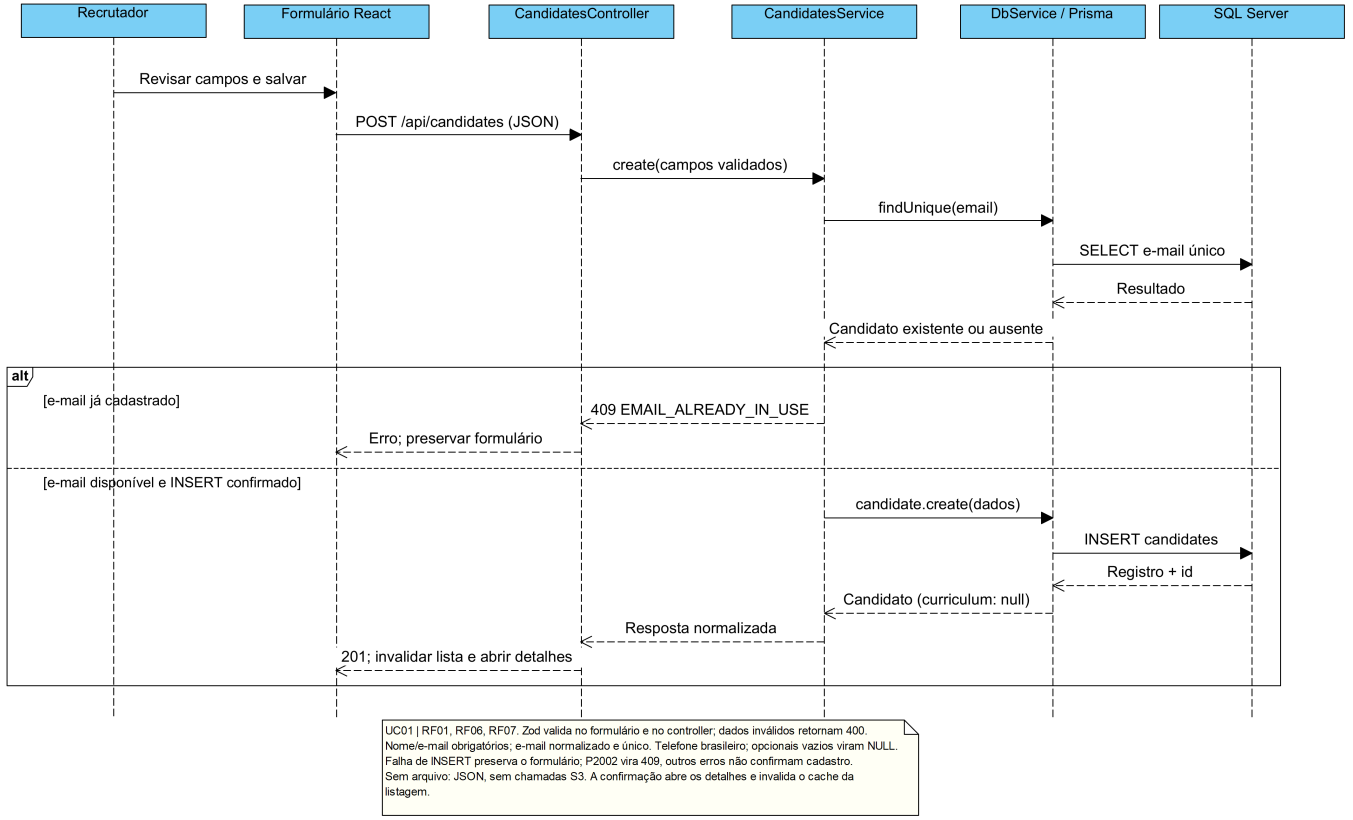


Figura 9: D06 – Cadastro manual

D07 – Consulta filtrada e paginada

A interface envia busca, filtros, ordenação e parâmetros de página. O serviço consulta itens e total com os mesmos filtros no SQL Server. A resposta contém resultados e metadados de paginação.

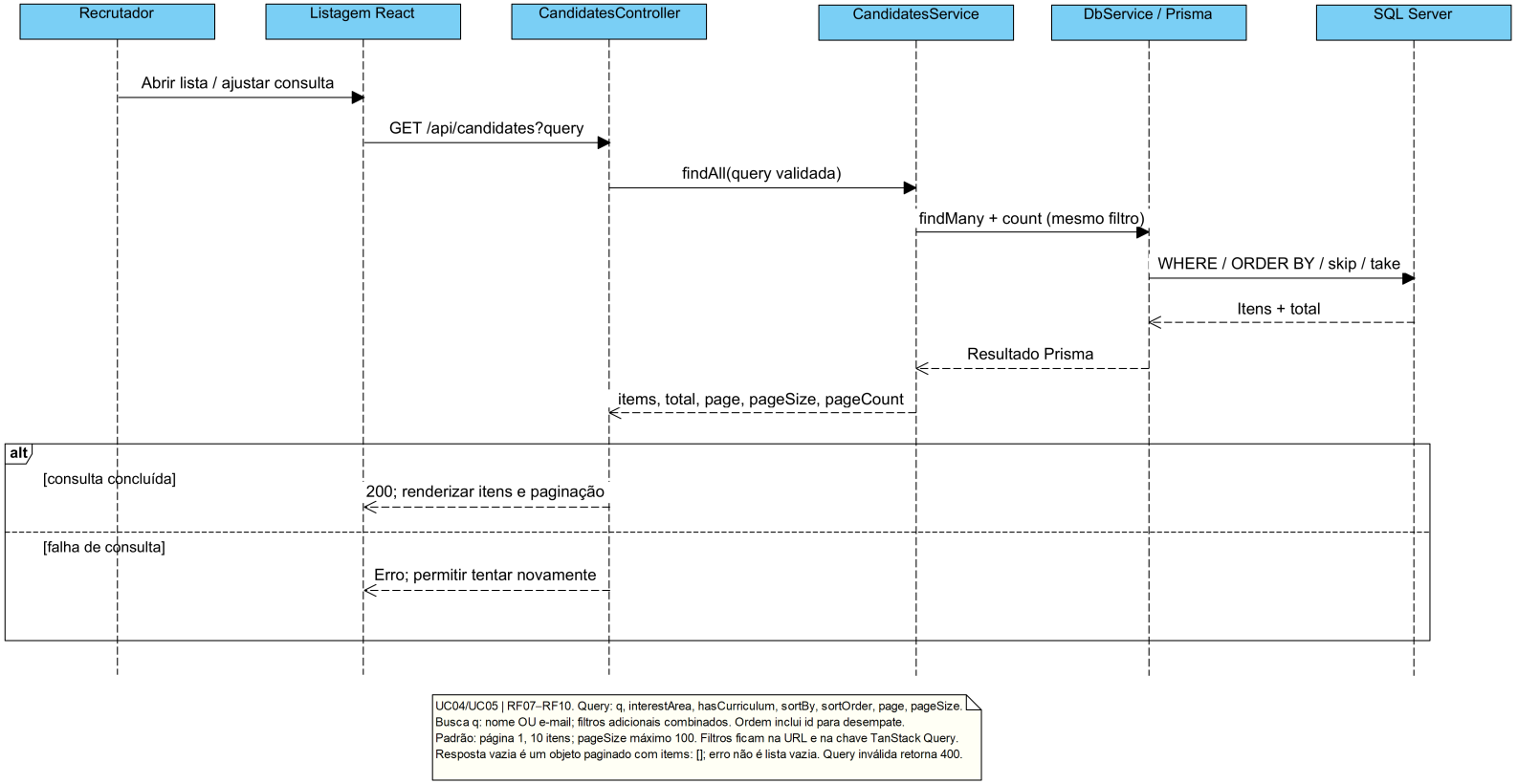


Figura 10: D07 – Consulta filtrada e paginada

D08 – Consulta de detalhes

A API consulta o candidato e retorna os campos cadastrados e os metadados do PDF, quando existente. A chave interna de armazenamento não é exposta. Um registro ausente recebe resposta específica.

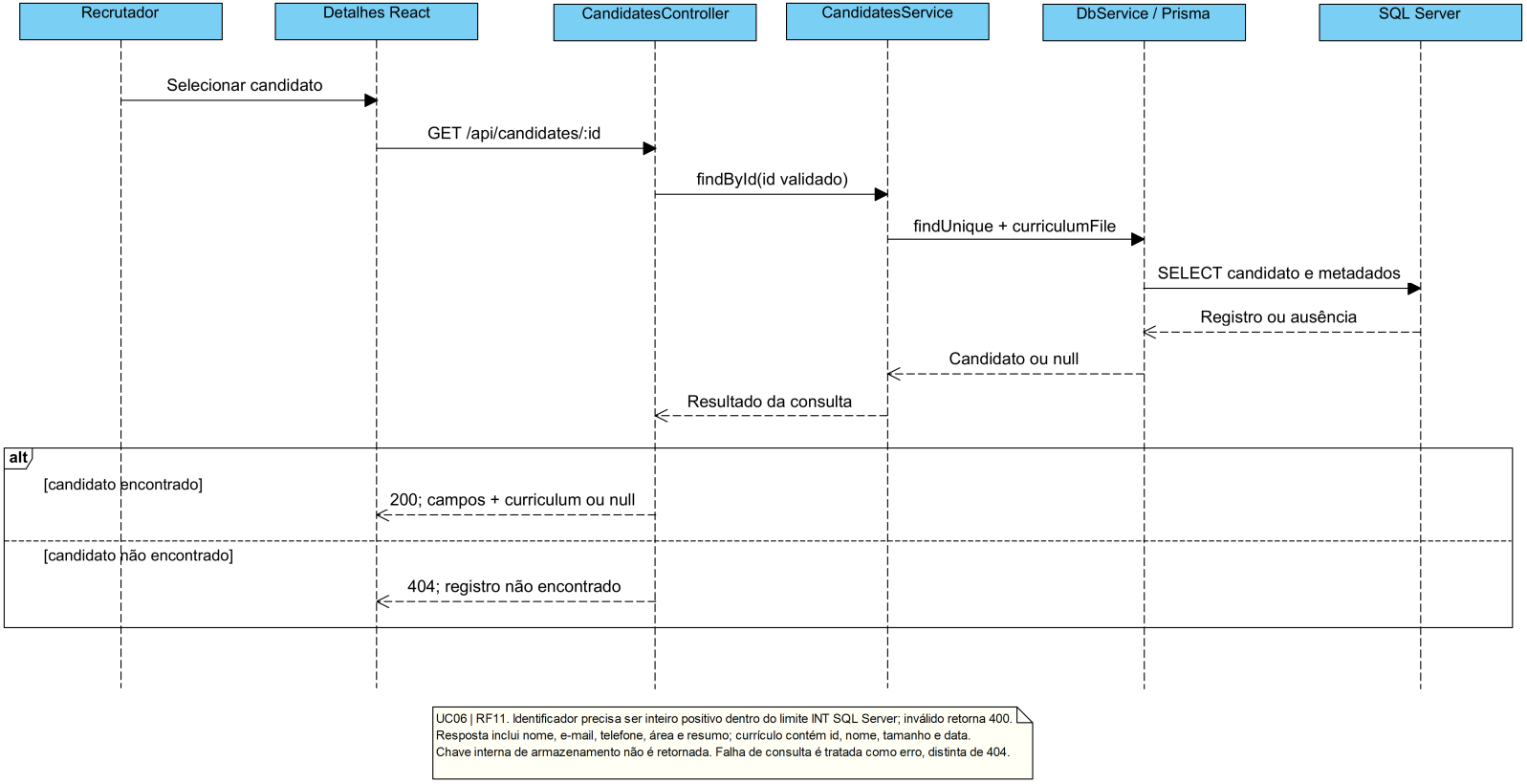


Figura 11: D08 – Consulta de detalhes

D09 – Cadastro com arquivo opcional

Guardar o PDF é uma escolha explícita após revisar os dados. Com arquivo, o backend faz upload e cria candidato e metadados; se SQL falhar, tenta remover o objeto enviado. Sem arquivo, o cadastro permanece disponível.

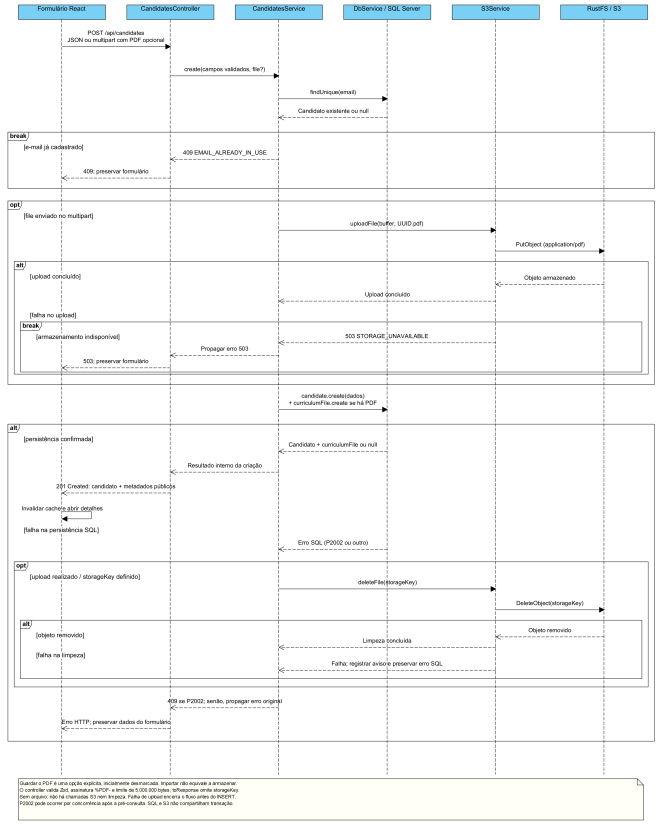


Figura 12: D09 – Cadastro com arquivo opcional

D10 – Download do currículo

O backend verifica candidato e metadados e retorna a URL pública configurada para o ambiente local. O frontend valida o protocolo HTTP/HTTPS e o navegador solicita o PDF diretamente ao armazenamento.

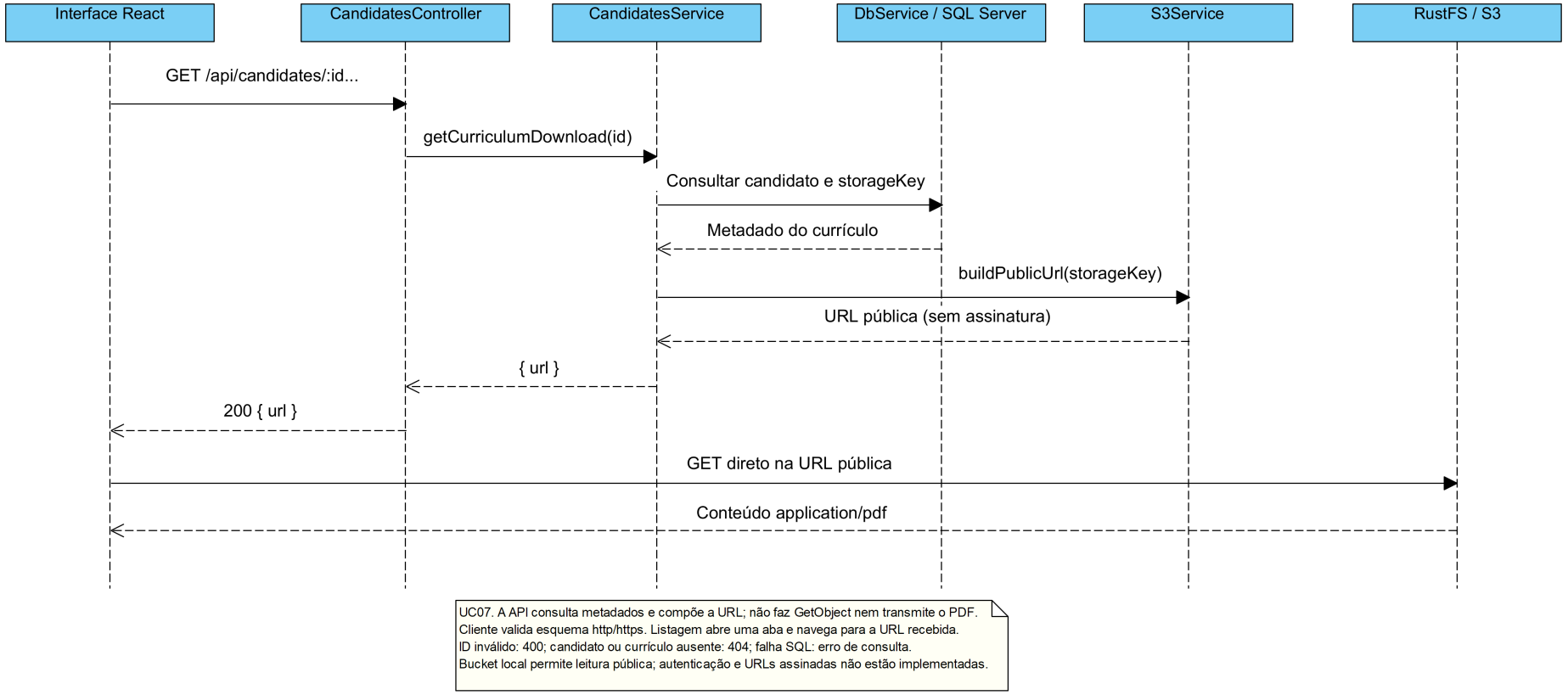


Figura 13: D10 – Download do currículo

D11 – Exclusão individual

A interface pede confirmação. A API consulta o candidato, remove o arquivo associado e depois o registro SQL. Uma falha na remoção do arquivo impede a etapa de exclusão no banco.

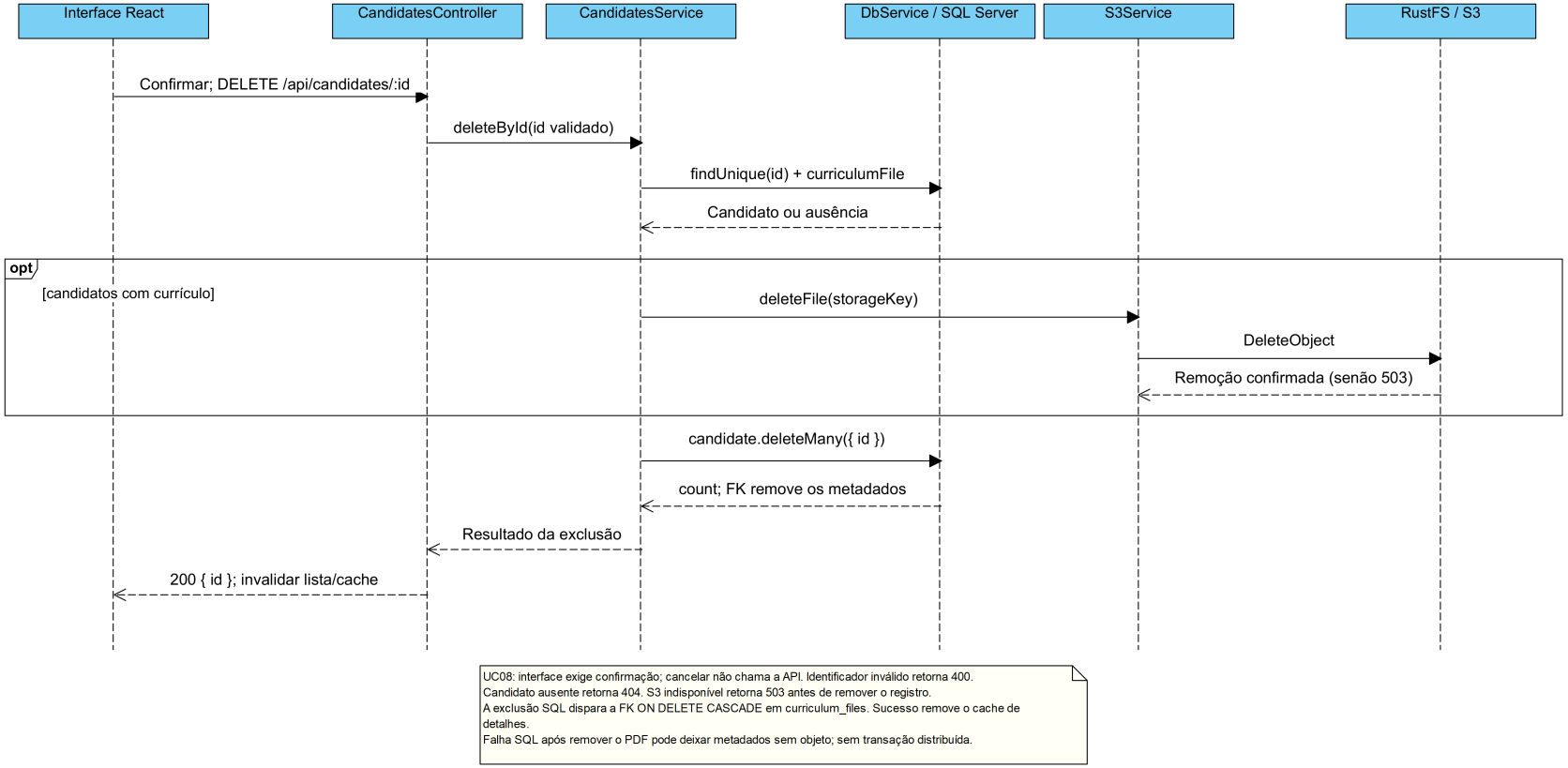


Figura 14: D11 – Exclusão individual

D12 – Exclusão em lote pela API

A operação valida até 100 IDs, remove duplicatas e considera os candidatos existentes. O serviço remove seus arquivos antes da etapa SQL e informa a quantidade excluída. Esse recurso é disponibilizado pela API.

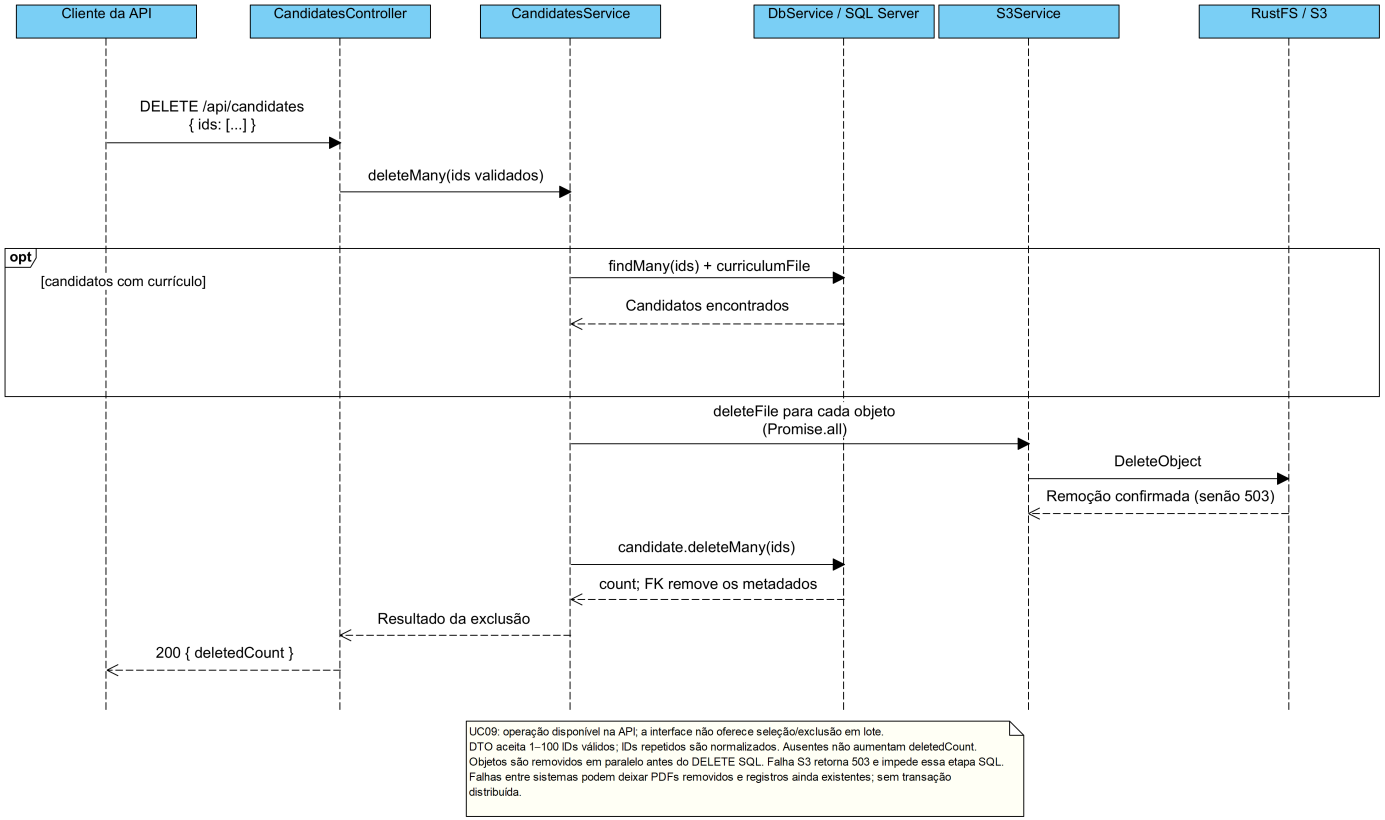


Figura 15: D12 – Exclusão em lote pela API

D13 – Descarte de alterações

O formulário confirma a saída quando há alterações. O usuário pode permanecer ou descartar os dados. Ao desmontar, a interface cancela a importação ativa; a leitura do PDF não exige limpeza de dados persistidos.

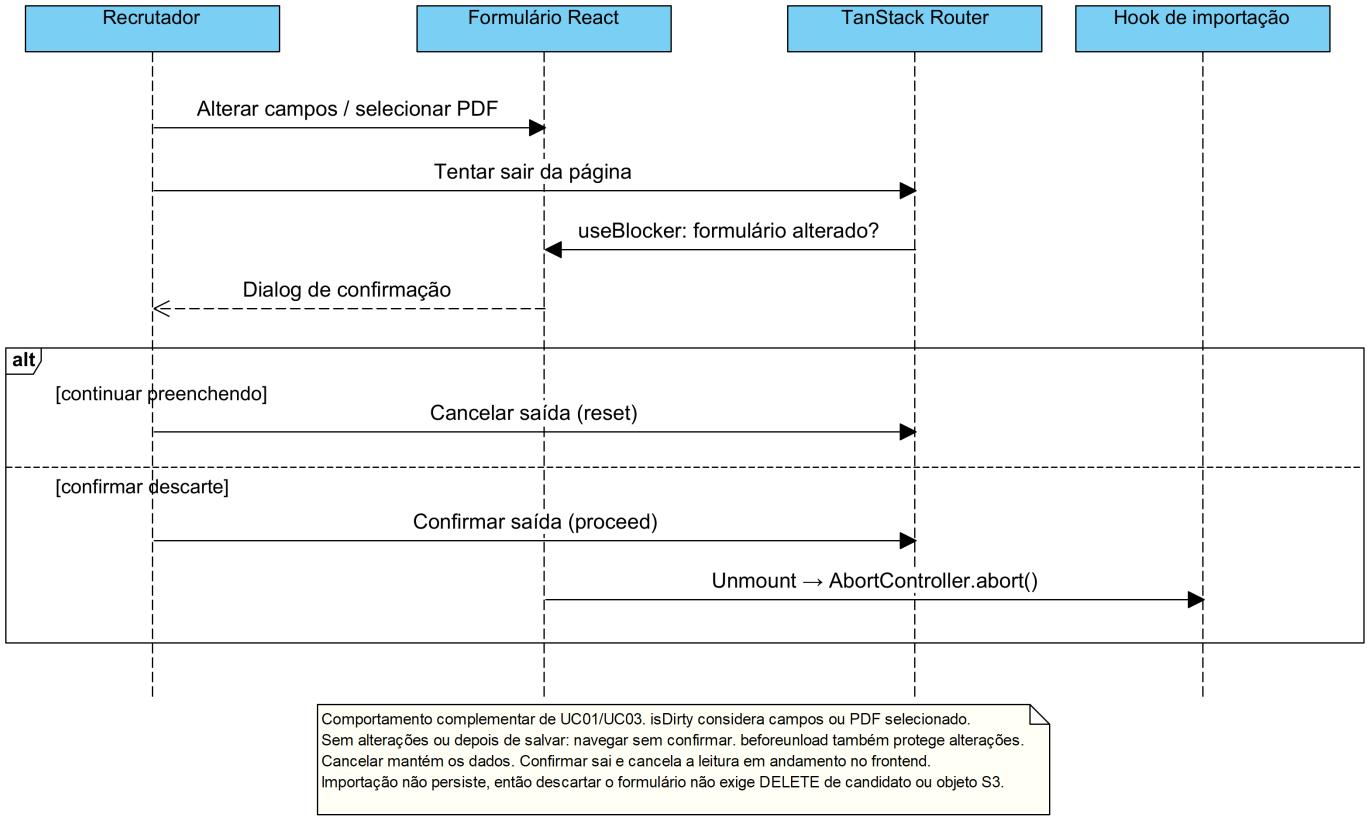


Figura 16: D13 – Descarte de alterações